

TRACE.PAY

FULL PROJECT BLUEPRINT

An Explainable Temporal Graph Platform for Evidence Aware Cyber Fraud Fund Tracing and Pre Payment Risk Advisory in India's UPI Ecosystem

Project vision • Research framing • Detailed architecture • Data model • APIs • Algorithms • Evaluation • Roadmap

Prepared for capstone planning and supervisor review

An Explainable Temporal Graph Platform for Evidence Aware Cyber Fraud Fund Tracing and Pre Payment Risk Advisory in India's UPI Ecosystem

Project vision • Research framing • Detailed architecture • Data model • APIs • Algorithms • Evaluation • Roadmap

Prepared for capstone planning and supervisor review

Document Guide

Part	Contents
1	Executive summary and project definition
2	Problem statement, research questions, objectives and scope
3	Users, roles, requirements and use cases
4	System architecture and component responsibilities
5	Technology stack and design decisions
6	Dataset strategy, preprocessing and privacy
7	Evidence model and provenance
8	Detection rules, temporal graph and path reconstruction
9	Payer interface and simulated payment workflow
10	Database schema and API contracts
11	Security, governance and report correction
12	Evaluation methodology and metrics
13	Frontend structure, repository layout and implementation roadmap
14	Testing, risks, deliverables, demo and final report outline

1. Executive Summary

Trace.Pay is a proposed investigator-support and payment-risk advisory platform for studying suspicious transaction patterns and reconstructing multi-hop fund flows in a UPI-inspired environment. It combines a payer-facing payment simulator with an investigator-facing console, both powered by a shared transaction intelligence engine.

The system represents accounts as graph nodes and recorded transfers as directed, time-stamped edges. It applies transparent, configurable rules to identify patterns that may merit review, including multiple incoming transfers from distinct counterparties, multiple outgoing transfers, and rapid onward movement. Every observed transfer retains a reference to its source record. Where the available data is incomplete, the platform explicitly identifies the gap rather than inventing a transfer or treating missing information as proof that no transfer occurred.

The payer interface scans a sandbox QR code, requests a recipient risk advisory, and allows the user to cancel or continue a simulated payment. The investigator console presents a graph, timeline, alert explanations, source references, and a summary of known and unknown portions of a reconstructed path.

Core principle A risk alert is not proof of fraud, guilt, or knowing participation. The prototype does not execute real UPI payments or access banking systems unless a separate, authorised integration is established.

1.1 One-line project definition

Trace.Pay connects explainable payment-risk assessment with source-linked temporal fund-flow reconstruction while making evidence gaps visible.

1.2 Product structure

Module	Purpose	Users
Trace.Pay shared engine	Ingestion, graph, detection, tracing, risk assessments	All authorised roles
Payer Risk Interface	Sandbox QR, recipient check, advisory, simulated payment	Demo payer / participant
Investigator Console	Graph, timeline, alerts, paths, source records, gaps	Analyst / investigator
Review and administration	Report review, corrections, role management, audit	Reviewer / administrator

2. Problem Definition and Research Framing

2.1 Background and problem

Cyber-enabled payment fraud can involve several accounts and transfers over short periods. Investigators may need to combine transaction exports, account statements, complaint references, timestamps, and information from multiple sources. Records can overlap, use inconsistent formats, contain duplicates, or stop before the observed trail is complete. A graph visualisation alone does not solve these issues: the system must distinguish transfers supported by source records from analytical hypotheses and unknown portions.

2.2 Problem statement

There is a need for a transparent prototype that ingests permitted transaction records, identifies explainable patterns, reconstructs time-constrained multi-hop paths, preserves source provenance for every observed transfer, and reports where available evidence is incomplete. The same risk service should support a sandbox payer interface before a simulated payment is confirmed, without disclosing restricted investigation details.

2.3 Primary research question

Research question Can a shared temporal-graph engine that links explainable risk alerts to source records and explicitly represents evidence gaps improve the verifiability and transparency of multi-hop transaction reconstruction under controlled evaluation?

2.4 Supporting questions

- How accurately does the system reconstruct known paths from datasets with ground truth?
- Can it identify deliberately introduced gaps without fabricating unsupported transfers?
- Can every displayed observed edge be traced to a source row or source-document reference?
- How quickly can a newly accepted event update the graph and risk assessment?
- Do users understand the difference between a risk advisory, an unverified report, and a verified finding?
- How does the system behave with duplicate, delayed, out-of-order, or incomplete records?

2.5 Objectives

- Design a temporal transaction graph preserving transaction identity, direction, time, amount, and provenance.
- Implement transparent detection rules with human-readable explanations.
- Build time-respecting multi-hop path reconstruction and path-level evidence summaries.
- Create a shared risk service used by both interfaces.
- Generate controlled datasets with known paths, labels, and deliberate gaps.
- Evaluate path correctness, provenance coverage, gap detection, alert latency, and warning comprehension.
- Document privacy, access-control, correction, and audit requirements.

2.6 Scope boundaries

In scope	Out of scope for the MVP
Synthetic and verified, permitted public datasets	Unauthorised access to bank or law-enforcement systems
Sandbox QR codes and simulated payments	Executing real UPI payments
Explainable rules and graph-based path reconstruction	Automated declarations of guilt or intent
Source references, gap reports, correction workflow	Guaranteeing fund recovery or complete tracing
Role-based access and audit logs	A public searchable blacklist of named people/accounts

3. Users, Roles and Requirements

3.1 Roles and permissions

Role	Allowed actions	Restrictions
Payer/demo user	Scan sandbox QR, request advisory, cancel or simulate payment	Cannot access restricted case details or other users' reports
Analyst/investigator	Search permitted cases, inspect graph, timeline, alerts, sources	Cannot verify reports without reviewer permission
Reviewer	Review reports, request information, verify/reject/dispute, process corrections	Every decision requires a reason and audit entry
Administrator	Manage users, roles, configuration and health	Admin access must not bypass audit or evidence rules
Research evaluator	Run labelled test scenarios and compute metrics	No access to identifiable records unless authorised

3.2 Functional requirements

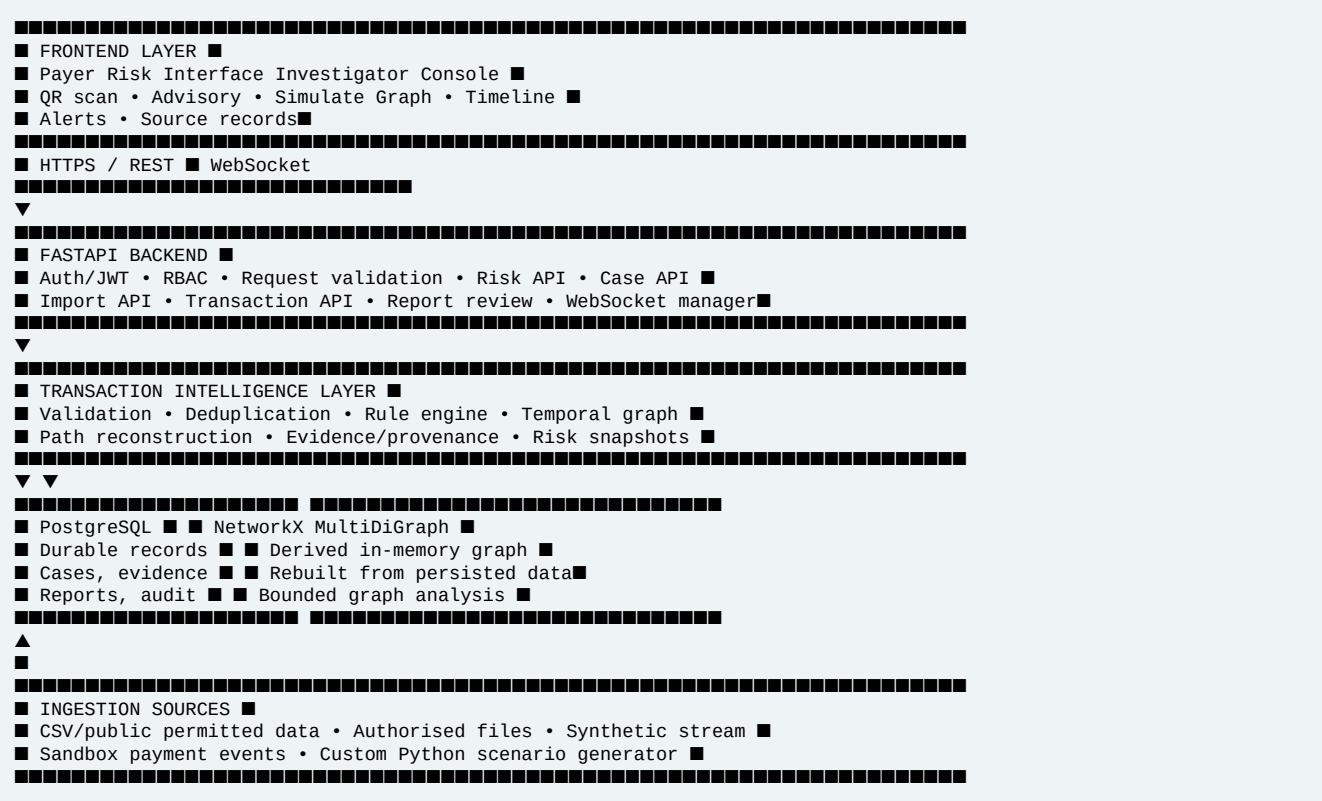
- Import a permitted file, map columns, preview rows, validate fields, and show rejected rows.
- Store validated transactions and source-record references.
- Create a directed temporal graph and support bounded path queries.
- Run configurable fan-in, fan-out, rapid pass-through, and cross-case-overlap rules.
- Show an explanation and supporting transaction IDs for each alert.
- Expose a minimal recipient risk advisory through a dedicated endpoint.
- Support sandbox QR scans and simulated payment attempts only.
- Stream accepted synthetic events to authorised connected clients.
- Support report review, correction/appeal history, and audit events.
- Export a redacted case summary with source references, coverage, and limitations.

3.3 Non-functional requirements

Requirement	Design target
Correctness	Known paths and gaps pass deterministic test cases
Traceability	Every observed edge links to a valid source record
Security	Backend-enforced permissions and least privilege
Privacy	Minimise, pseudonymise, restrict and redact sensitive fields
Performance	Bounded graph queries; measure rather than assume latency
Reliability	Database is durable source of truth; graph can be rebuilt
Usability	Clear distinction between source facts, alerts, and unknowns
Reproducibility	Versioned rules, documented data, repeatable scenarios

4. Detailed System Architecture

4.1 Logical architecture



4.2 Layer responsibilities

Layer	Responsibilities	Technology
Presentation	Forms, QR scan, graph, timeline, alerts, advisory, report screens	React, Vite, Tailwind, Cytoscape.js
API and identity	REST, WebSocket auth, schema validation, role checks	FastAPI, Pydantic, JWT
Ingestion	File parsing, field mapping, validation, deduplication	Pandas, validation services
Domain services	Rules, graph updates, path search, evidence, report lifecycle	Python modules
Persistence	Transactions, accounts, cases, sources, assessments, audit	PostgreSQL, SQLAlchemy, Alembic
Graph analysis	Temporal traversal, path reconstruction, graph payloads	NetworkX MultiDiGraph
Live updates	Publish accepted transactions and alerts	WebSockets; optional Redis later
Operations	Local setup, configuration, logs, tests	Docker Compose, pytest

4.3 End-to-end transaction flow

- A permitted file or synthetic event enters the ingestion service.
- Validate required fields, normalize timestamps, pseudonymise identifiers where required, and check event idempotency.
- Persist the transaction and source reference in PostgreSQL.
- Update the graph projection from the persisted event.
- Evaluate rules over the affected account neighbourhood and time window.
- Persist alerts and a versioned risk-assessment snapshot.

- Broadcast a minimal event to authorised WebSocket subscribers.
- Refresh the investigator graph and timeline; the payer interface obtains an advisory through a risk-check request.

4.4 Reliability decisions

- Use a unique event_id or idempotency key to prevent duplicate effects.
- Record transaction time and ingestion time separately to handle late-arriving records.
- On startup, rebuild the graph from persisted records or a documented snapshot.
- Version rules and record the data-as-of timestamp used for each risk assessment.
- Use bounded graph queries, hop limits, time windows, and result caps.
- Start with a modular monolith. Add Redis, queues, or separate services only if required.

5. Technology Stack and Rationale

Technology	Purpose	Implementation note
Python	Core backend and data analysis	Keep domain logic in service modules
React + Vite	Web interfaces	Shared components and typed API client
Tailwind CSS	Styling	Consistent responsive layout
Cytoscape.js	Graph visualisation	Selection, layouts, filtering and edge details
FastAPI	REST and WebSocket backend	Version API routes and document OpenAPI
Pydantic	Schema validation	Validate all client and ingestion inputs
Pandas	Batch processing	Preview, normalize, validate and report data quality
PostgreSQL	Durable storage	Constraints, indexes, transactions
SQLAlchemy/Alembic	ORM and migrations	Version schema changes
NetworkX MultiDiGraph	Graph analysis	Multiple transfers between the same accounts
JWT + RBAC	Authentication and permissions	Enforce server-side, not only in the UI
Docker Compose	Local reproducible setup	Frontend, backend, database
pytest	Testing	Unit, integration and API tests

Avoid premature complexity A graph neural network, graph database, Kafka, or Kubernetes is not required to prove the core contribution. Add ML only if suitable labels, data volume, and a meaningful baseline are available.

6. Dataset Strategy and Governance

6.1 Recommended data tracks

Data source	Use	Limitations
AMLSim-generated transactions	Controlled network patterns and ground truth	Synthetic patterns may not match Indian UPI behaviour
Custom Python scenarios	Known multi-hop paths, gaps, duplicates, cross-case overlaps	Scenario design can bias results
Public transaction dataset	Supplementary experiments if provenance/licence/fields are suitable	Do not call it Indian UPI data unless its source establishes that
Authorised original statements/exports	Potential validation against source records	Requires permissions, minimisation, secure storage and access controls
Published aggregate statistics	Problem motivation and context	Cannot validate row-level model performance

6.2 Dataset verification checklist

- Record exact title, publisher, stable URL, retrieval date, version, licence and citation.
- Document whether the data is real, synthetic, anonymised, or mixed.

- Document every field and the source meaning of every label.
- Check direct identifiers, quasi-identifiers, account details, and confidentiality restrictions.
- Keep raw input immutable and record all transformation steps.
- Separate training/test scenarios by account group, scenario, or time where appropriate to avoid leakage.
- Do not publish raw statements or restricted case records in public repositories or demos.
- Do not assume possession of a file means permission to use it for research.

6.3 Processing pipeline

```

Raw file/event
↓
Schema and file validation
↓
Field mapping and type coercion
↓
Timestamp parsing and timezone policy
↓
Pseudonymisation where required
↓
Duplicate / idempotency check
↓
Quality report + rejected-row quarantine
↓
Persist transaction and source-record reference
↓
Graph projection → rules → assessment snapshot

```

6.4 Canonical transaction schema

Field	Type	Meaning
transaction_id	UUID/string	Internal unique identifier
source_transaction_ref	String	Original source reference, where available
event_id	String	Idempotency key
sender_account_id	String	Pseudonymous sender reference where appropriate
recipient_account_id	String	Pseudonymous recipient reference where appropriate
amount	Decimal	Positive amount; preserve source semantics
currency	String	Currency code where known
transaction_time	Timestamp	Source transaction time and timezone policy
ingested_at	Timestamp	System ingestion time
source_type	Enum	Synthetic, public dataset, authorised statement, etc.
evidence_record_id	UUID/string	Link to source metadata
data_quality_status	Enum	Accepted, accepted with warning, rejected

6.5 Scenario suite

- Legitimate baseline activity.
- Fan-in: distinct senders transfer to one recipient in a defined time window.
- Fan-out: one sender transfers to multiple recipients.
- Rapid pass-through: incoming transfer followed by outgoing transfers.
- Known multi-hop path such as $A \rightarrow B \rightarrow C \rightarrow D$.
- Cross-case overlap involving a pseudonymous account.
- Missing source record introduced deliberately to test gap reporting.
- Duplicate and out-of-order event replay.
- Benign lookalike pattern to test false positives.
- Disputed report and correction workflow.

7. Evidence Model and Provenance

7.1 Core concepts

Concept	Meaning	Representation
Observed transaction	A validated transfer exists in a source record	Directed, timestamped edge with source reference
Inferred relationship	Analytical hypothesis not directly recorded as a transfer	Separate typed analytical link, never a confirmed transfer
Evidence gap	Available records cannot establish a portion of the trail	Gap annotation/path coverage status, not a fabricated edge
Unverifiable claim	Claim lacks sufficient support in available sources	Claim/report with status and rationale
Detection alert	Rule identifies a pattern that merits review	Separate alert linked to supporting transactions

7.2 Path-level evidence summary

Do not force every uncertainty into a single edge label. A path result should list observed edges, any separate inferred links, data coverage, missing intervals, and the source references. Suggested path summaries include Complete within supplied records, Partially supported, Contains inferred links, and Insufficient data. These labels describe support in the dataset, not criminality.

```

Example path result
Start: SANDBOX-A
End: SANDBOX-D
Observed edges: TX-101, TX-118
Inferred links: none
Coverage: partial
Gap: no source record available for the later interval
Interpretation: observed movement is supported up to SANDBOX-C;
further movement is unknown from the available records.
Data as of: timestamp
Algorithm/rule version: recorded version

```

7.3 Provenance fields

- Source record identifier and source type.
- Dataset/file ID, version, and import batch.
- Row number or stable source locator where possible.
- File hash or record-bundle hash where appropriate.
- Ingestion timestamp, validation status, and transformation history.
- Pseudonymisation method/version identifier where applicable.
- Audit events for source access, review, and export.
- Correction, supersession, or dispute history.

7.4 Claims the system must avoid

- A source record can support a recorded transfer, but does not itself prove fraud or intent.
- Do not attribute a particular victim's exact money to a downstream account after commingling unless the evidence and method support that attribution.
- Do not infer knowing participation from graph topology alone.
- An absent statement is a data gap, not proof that no transfer occurred.
- Do not call outputs court-ready or legally conclusive without qualified review.

8. Detection Rules and Temporal Fund Tracing

8.1 Explainable rules

Rule family	Illustrative pattern	Explanation	Caution
Fan-in	Distinct senders pay one recipient within a configured window	Multiple distinct counterparties transferred funds in the selected period	May be legitimate collection activity
Fan-out	One sender pays several recipients	This account sent funds to multiple counterparties	Not proof of laundering
Rapid pass-through	Incoming transfer followed by outgoing transfer(s) within an interval	Recorded incoming activity was followed by onward movement	Other transactions or balances may be missing
Cross-case overlap	Same pseudonymous identifier occurs in separate permitted cases	Identifier overlaps in the supplied dataset	Matching must be reliable and authorised
Velocity change	Activity differs from documented baseline	Count or amount changed relative to the configured baseline	Needs sufficient baseline data

8.2 Alert object

```
{
  "alert_id": "ALT-00042",
  "rule_id": "RAPID_PASS_THROUGH_V1",
  "rule_version": "1.0",
  "severity": "REVIEW",
  "explanation": "Incoming activity was followed by onward transfers.",
  "supporting_transaction_ids": ["TX-101", "TX-118"],
  "data_as_of": "timestamp",
  "evidence_status": "PARTIAL",
  "limitations": ["Available records do not establish the full downstream path."]
}
```

8.3 Temporal graph

Use a NetworkX MultiDiGraph because the same sender and recipient can have multiple transfers. Each edge should include transaction ID, amount, currency, transaction time, source reference, and data-quality metadata. Nodes should use pseudonymous identifiers in the research environment.

8.4 Time-respecting path algorithm

- Validate the start account, time window, and maximum hop count.
- Traverse observed outgoing edges within the requested time window.
- Require non-decreasing transaction time along candidate paths.
- Avoid repeated nodes unless cycle analysis is explicitly enabled.
- Retain source references for every observed edge.
- Represent missing coverage as a gap annotation, not a fabricated transfer.
- Return bounded results with path-level evidence summaries and truncation metadata.

8.5 Amount and commingling limitations

The sum of incoming and outgoing transactions is not a verified account balance. Records may omit opening balances, cash activity, fees, reversals, unrelated transactions, or other accounts. Report observed flows rather than claiming a specific victim's money moved to a downstream account unless the data and method support that conclusion.

8.6 Optional machine learning

Machine learning is optional. Add it only if the selected dataset has meaningful labels, enough volume, a leakage-resistant evaluation split, and a clear baseline. A GNN is not required for the evidence-aware contribution. If used, compare it against simple rules and report false positives, calibration, and domain limitations.

9. Payer Risk Interface and Payment Simulation

9.1 User journey

- User opens the sandbox and scans a QR code created for the demonstration.

- The QR resolves to an internal sandbox recipient ID, not a live payment instruction.
- Frontend submits recipient ID and optional simulated amount to the risk-check endpoint.
- Backend returns minimal advisory, reasons, status, limitations, and data-as-of timestamp.
- User chooses Cancel or Continue Simulation; no real payment is initiated.
- If continued, the simulated payment is persisted and enters the same event pipeline.
- The investigator console receives the event and shows the resulting graph or alert update.

9.2 Advisory statuses

Status	Meaning	UI guidance
No known warning	No matching warning found in available prototype data	State that this does not guarantee safety
Insufficient information	System cannot provide a meaningful assessment	Do not show as 'safe'
Unverified report	A report exists but is not verified	Clearly label as unverified
Under review	A report or pattern is being assessed	Show limited context
Verified warning	A report meets documented review criteria	Show concise reasons and correction route
Disputed/corrected	A previous report is challenged or changed	Show current status and preserve history

9.3 Privacy and sandbox boundary

Do not expose identities, private case details, complainant information, raw statements, or the full graph to the payer interface. Enforce this on the backend. Use sandbox QR codes and simulated payment confirmation. Never collect UPI PINs, OTPs, bank credentials, or payment passwords. Do not claim integration with Google Pay, PhonePe, NPCI, or any bank without a separately authorised integration.

10. Database Design

Table	Important fields	Purpose
users	id, username/email, credential reference, active	Identity
roles/user_roles	user_id, role_id, role_name	Permissions
cases	id, case_ref, status, created_by, created_at	Case grouping
accounts	id, pseudonymous_ref, source_namespace	Graph nodes
transactions	id, event_id, sender_id, recipient_id, amount, currency, transaction_time, source_record_id	Observed transfers
source_files	id, safe_filename, source_type, version, hash, licence note	Dataset provenance
source_records	id, source_file_id, source_locator, validation_status	Record provenance
import_batches	id, status, accepted_count, rejected_count, timestamps	Ingestion history
alerts	id, case_id, rule_id, rule_version, severity, explanation	Detection outputs
alert_transactions	alert_id, transaction_id	Alert support links
risk_assessments	id, recipient_id, status, reasons, data_as_of, rule_version	Reproducible advisories
fraud_reports	id, recipient_ref, allegation, status, submitted_by, reviewed_by	Report lifecycle
report_reviews	id, report_id, decision, reason, reviewer_id, reviewed_at	Review history
correction_appeals	id, report_id, type, reason, status, resolution	Correction route
audit_events	id, actor_id, action, target, timestamp, metadata	Accountability
simulated_payments	id, user_id, recipient_id, amount, outcome, assessment_id	Sandbox attempts

10.1 Integrity and indexing

- Unique constraint on event_id within the relevant source namespace.
- Foreign keys from transactions to accounts and source records.
- Use fixed precision Decimal/numeric types for money.
- Index sender, recipient, transaction time, case ID, source record, and alert time.
- Use migrations rather than manually changing database schemas.
- Define retention and deletion rules for test data and any authorised sensitive data.

11. API Design

Method / route	Purpose	Access
POST /api/v1/auth/login	Authenticate and issue token	Login route with rate limits
POST /api/v1/imports	Create import batch	Analyst/admin
POST /api/v1/imports/{id}/validate	Validate and preview	Analyst/admin
POST /api/v1/imports/{id}/commit	Commit accepted rows	Analyst/admin
GET /api/v1/imports/{id}	Import summary and rejected rows	Authorised case role
POST /api/v1/transactions	Ingest one test transaction	Ingestion service/admin
GET /api/v1/transactions	Search permitted transactions	Investigator
GET /api/v1/accounts/{id}/timeline	Account activity over a time window	Investigator
GET /api/v1/graph/paths/{account_id}	Reconstruct time-respecting paths	Investigator
GET /api/v1/graph/neighbourhood/{account_id}	Bounded graph neighbourhood	Investigator
GET /api/v1/alerts	List alerts	Investigator
GET /api/v1/alerts/{id}	Alert explanation and supporting records	Authorised case role
GET /api/v1/evidence/{transaction_id}	Source reference and provenance	Authorised case role
POST /api/v1/risk/check	Minimal recipient advisory	Payer/demo user
POST /api/v1/reports	Submit a report	Authenticated user
POST /api/v1/reports/{id}/review	Review report	Reviewer
POST /api/v1/reports/{id}/appeals	Correction/appeal request	Eligible user
POST /api/v1/simulated-payments	Record sandbox payment	Payer/demo user
GET /api/v1/audit-events	Search audit events	Auditor/admin
WS /ws/live	Receive authorised updates	Authenticated subscriber

11.1 Example risk-check response

```
{
  "recipient_ref": "SANDBOX-RECIPIENT-104",
  "assessment_status": "UNDER_REVIEW",
  "risk_level": "REVIEW",
  "reasons": ["Multiple distinct counterparties transferred funds.",
    "Recorded incoming activity was followed by onward transfers."],
  "report_status": "UNVERIFIED",
  "evidence_scope": "SYNTHETIC",
  "data_as_of": "timestamp",
  "payment_executed": false,
  "limitations": ["Prototype advisory; not a determination of fraud.",
    "No live UPI payment was initiated."]
}
```

11.2 API implementation rules

- Use Pydantic request and response models; never trust client-supplied roles.
- Return structured validation errors and stable error codes.
- Use pagination, bounded time windows, hop limits, and query timeouts.
- Do not expose source-file paths or raw statements to the payer interface.
- Rate-limit login, report submission, and recipient checks.
- Authenticate WebSocket connections and authorise subscriptions.
- Version routes and maintain OpenAPI documentation.

12. Security, Privacy and Governance

12.1 Minimum safeguards

- Collect only fields needed for the research question.
- Pseudonymise account identifiers in development and demonstrations.
- Keep secrets in environment variables, not source control.
- Use HTTPS for shared deployments and secure database credentials.
- Hash local passwords with a suitable password hashing algorithm.
- Enforce role and case-level permissions on the backend.
- Audit imports, exports, report decisions, role changes, and administrative actions.
- Do not log tokens, raw statements, or sensitive payloads.
- Restrict exports and apply redaction by default.
- Define backup, retention, deletion, and incident response procedures.
- Use synthetic data for public demos and screenshots.
- Obtain required institutional approval for human-participant studies.

12.2 Report lifecycle

```

SUBMITTED → UNDER_REVIEW
■■■ VERIFIED (basis and reviewer recorded)
■■■ REJECTED (reason recorded)
■■■ NEEDS_INFORMATION
■■■ DISPUTED / APPEAL_REQUESTED
↓
REVIEW_RESOLUTION
↓
Current status updated; history retained

```

A verified report should have a documented basis and an authorised reviewer. Corrections must update the current presentation without erasing the audit history.

12.3 Legal and institutional note

This is a technical planning document, not legal advice. Before processing original bank statements, police case records, or identifiable financial data, confirm permissions, purpose limitations, institutional policy, applicable Indian law, and data-sharing conditions with the supervisor and authorised data custodian. Possession of a file does not automatically establish permission to use it for a capstone.

12.4 Threat model

Threat	Impact	Mitigation
Unauthorised case access	Sensitive records exposed	RBAC, case scoping, denial tests
Malformed or oversized CSV	Bad data/resource exhaustion	Schema allowlist, size limits, safe parsing
Duplicate event replay	Inflated counts and alerts	Idempotency key and unique constraint
False/malicious report	Unfair warning	Review, evidence basis, appeal and correction
Sensitive data in logs	Privacy breach	Redacted structured logging
Stale advisory	Outdated information	Data-as-of timestamp and refresh policy
Graph explosion	Slow queries or denial of service	Hop/time limits, caps and timeouts
Insider misuse	Unauthorised searches/exports	Least privilege and audit review

13. Evaluation and Experimental Design

13.1 Principle

Test only claims the system can support. Synthetic data provides known ground truth but does not establish real-world performance by itself. Report results separately for synthetic scenarios, public datasets, and any authorised real-world records.

13.2 Experimental tracks

Track	Data	Question
Functional correctness	Hand-authored cases	Are paths, alerts and gaps correct?
Synthetic benchmark	AMLSim + custom labelled scenarios	How well are patterns detected and paths reconstructed?
Missing-data robustness	Versions with selected records removed	Are gaps reported without fabricated edges?
Event-stream behaviour	Duplicates and late-arriving events	Are updates timely and idempotent?
Public dataset test	Verified suitable dataset, if available	What transfers, and what are the domain limitations?
Usability study	Consenting participants, if approved	Do users understand advisories and evidence limitations?

13.3 Metrics

Metric	Definition / use
Precision	True positive alerts / all positive alerts, where labels exist
Recall	True positive cases / all labelled positive cases
False positive rate	False positive cases / all actual negative cases
Path edge accuracy	Correct reconstructed observed edges / expected observed edges
Exact path match	Share of cases where returned path exactly matches expected sequence
Provenance coverage	Displayed observed edges with valid source references
Gap detection rate	Introduced gaps correctly identified / all introduced gaps
Unsupported-edge rate	Displayed transaction edges unsupported by source records
Alert latency	Time from accepted event to persisted alert/UI update
Idempotency correctness	Whether event replay creates duplicate effects
Warning comprehension	Task accuracy or questionnaire results
Time to verify	Time needed to locate source records supporting a path

13.4 Baselines and ablations

- Baseline A: simple rule alerts without multi-hop graph reconstruction.
- Baseline B: graph reconstruction without explicit evidence-gap reporting.

- Proposed method: graph reconstruction plus source provenance and explicit gap reporting.
- Optional ablation: remove temporal ordering and measure changes in path correctness.
- Use the same scenarios and documented test protocol for all comparisons.

13.5 Candidate engineering acceptance criteria

These are proposed targets to agree with the supervisor, not results.

Area	Candidate criterion
Provenance	Every observed edge in controlled tests links to a valid source record.
Unsupported edges	No transaction edge is fabricated to bridge an evidence gap.
Known paths	Hand-authored functional paths match ground truth.
Gap handling	All explicitly seeded gaps in functional tests are surfaced.
Idempotency	Replaying an event ID does not create a duplicate transaction.
Alert explainability	Each alert includes rule ID/version, explanation, and supporting IDs.
Role enforcement	Restricted endpoints deny unauthorised roles.
Sandbox boundary	No real payment instruction or UPI PIN/OTP is requested.

13.6 Pilot size clarification

Fifty simulated accounts are not the same as fifty human participants. A fifty-account network can support initial end-to-end tests, but a performance study should include many scenarios and varied graph sizes. A fifty-person usability study requires a separate protocol and any necessary approval.

14. Frontend Information Architecture

14.1 Payer interface

Screen	Main elements
Welcome	Sandbox notice, prototype disclaimer, start
QR scan	Camera permission and manual sandbox ID fallback
Recipient check	Recipient reference, amount, check button
Risk advisory	Status, reasons, data-as-of, limitations, cancel/continue
Simulation result	Simulated reference and clear no-real-payment notice
Report/feedback	Neutral form, privacy notice

14.2 Investigator console

Screen	Main elements
Overview	Import status, recent alerts, coverage, system health
Dataset import	Upload, field mapping, preview, validation, commit
Case workspace	Case summary, permitted sources, time range
Graph explorer	Nodes, directed edges, filters, edge details
Timeline	Chronological transfers, time window, late-event markers
Path results	Candidate paths, evidence status, gap annotations
Alert detail	Rule/version, explanation, supporting records, limitations
Source viewer	Redacted source reference and locator
Review workspace	Report status, reviewer decision, appeal history
Audit log	Authorised actions and exports

14.3 UI principles

- Use neutral wording such as 'risk advisory' and 'pattern detected'.
- Show evidence status with text and icons, not colour alone.
- Display timestamps, timezone, and data-as-of time.
- Separate source facts from rule-generated interpretations.
- Allow users to navigate from an alert to supporting records quickly.
- Bound graph complexity and provide a table alternative.

15. Repository Structure

```

trace-pay/
├── README.md
├── docker-compose.yml
├── .env.example
├── docs/
│   ├── architecture.md
│   ├── data-dictionary.md
│   ├── threat-model.md
│   └── evaluation-protocol.md
├── backend/
│   ├── alembic/
│   ├── app/
│   │   ├── main.py
│   │   └── api/v1/
│   │       ├── core/ # settings, security, logging
│   │       ├── db/ # session and base
│   │       ├── models/ # SQLAlchemy entities
│   │       ├── schemas/ # Pydantic contracts
│   │       └── services/
│   │           ├── ingestion.py
│   │           ├── validation.py
│   │           ├── graph_service.py
│   │           ├── path_tracing.py
│   │           ├── detection_rules.py
│   │           ├── evidence_service.py
│   │           ├── risk_service.py
│   │           └── report_review.py
│   ├── workers/
│   └── tests/
├── Dockerfile
├── frontend/
│   ├── src/
│   ├── components/
│   ├── pages/
│   ├── features/payer/
│   ├── features/graph/
│   ├── features/timeline/
│   ├── features/alerts/
│   ├── features/reports/
│   ├── api/
│   └── data/
│       ├── synthetic/
│       ├── public/
│       ├── README.md
│       └── scripts/
│           ├── generate_scenarios.py
│           ├── seed_demo.py
│           └── run_evaluation.py

```

Data hygiene Do not commit private statements, credentials, tokens, .env files, or restricted investigation data to Git. Keep raw-data folders in .gitignore unless the contents are safe and permitted to publish.

16. Implementation Roadmap

Phase	Work	Definition of done
1. Scope and data	Research question, data dictionary, source plan, threat model	Approved scope and documented scenarios
2. Foundation	Repo, Docker Compose, PostgreSQL, FastAPI, React shell	Local health checks pass
3. Ingestion	Preview, validation, pseudonymisation, idempotency	Good rows persist; bad rows are explained
4. Graph and tracing	Graph projection, temporal paths, provenance, gaps	Known paths and gap tests pass
5. Detection	Fan-in, fan-out, pass-through rules	Explainable alerts linked to records
6. Investigator UI	Graph, timeline, filters, evidence viewer	Analyst can trace alert to source
7. Payer simulator	Sandbox QR, advisory, simulated payment	No real payment; shared engine works
8. Live events	Synthetic producer, WebSockets, duplicate handling	Events update graph and UI
9. Governance	RBAC, review, appeals, audit	Access and correction tests pass
10. Evaluation	Scenarios, baselines, metrics, results	Reproducible evaluation report
11. Final delivery	Documentation, demo, presentation, polish	Clean setup and end-to-end demo

16.1 Sprint suggestion

Sprint	Focus	Deliverable
1	Scope, schema, environment, synthetic generator	Running skeleton and sample data
2	Ingestion, database, graph, path tracing	Source-linked paths and gap handling
3	Rules, risk API, investigator UI	Explainable alerts and graph view
4	QR simulator, WebSockets, RBAC, report lifecycle	Integrated two-interface demo
5	Tests, evaluation, documentation, presentation	Reproducible results and final report

If time is limited, prioritise reliable ingestion, source-linked path reconstruction, explicit gap handling, and the shared risk endpoint before optional ML or advanced visual polish.

17. Testing Strategy

17.1 Unit tests

- Timestamp parsing, timezone normalisation, and invalid values.
- Duplicate detection and event idempotency.
- Rule logic against hand-calculated examples.
- Temporal path traversal, hop limits, and cycles.
- Evidence summaries and gap reporting.
- Pseudonymisation consistency.
- Minimal payer advisory response.

17.2 Integration tests

- Import a file and verify database and graph consistency.
- Ingest a transaction and verify a relevant alert is stored.
- Run a risk check and verify its data-as-of time and rule version.
- Submit, review, dispute, and correct a report; verify audit history.
- Verify restricted users cannot access source records or reviewer endpoints.

- Reconnect WebSocket clients and refresh from durable state.

17.3 End-to-end test

1. Seed synthetic accounts A, B, C, D.
2. Ingest $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$ with known timestamps.
3. Verify all three edges have source references.
4. Query a path from A and compare it with ground truth.
5. Test a separate dataset with a deliberately missing source record.
6. Confirm the path result reports unknown downstream coverage.
7. Trigger a rule and inspect its explanation and supporting IDs.
8. Scan B's sandbox QR and request a risk advisory.
9. Simulate a payment and verify the event appears in the graph/audit log.
10. Replay the same event ID and verify it is not counted twice.

18. Risks and Limitations

Risk / limitation	Why it matters	Mitigation
No suitable public UPI dataset	Limits external validation	Verify sources; use controlled synthetic scenarios
Synthetic-to-real gap	Synthetic performance may not generalise	Report data tracks separately
False positives	Legitimate activity may resemble suspicious patterns	Explain rules, evaluate, allow review/correction
Incomplete statements	Trail may end before actual endpoint	Report coverage gaps; do not infer missing transfers
Incorrect entity matching	Different accounts could be wrongly linked	Conservative matching and corroboration
Commingled funds	Victim-specific attribution may be unsupported	Describe recorded flows, not unsupported ownership
Stale advisory	Risk status may have changed	Show data-as-of and refresh
Sensitive data exposure	Potential harm and compliance issue	Minimise, pseudonymise, encrypt, restrict, audit
Scope creep	Core contribution may remain unfinished	Modular monolith and staged milestones
Unverified novelty claim	Reviewers may challenge the gap	Document literature search and narrow claims

18.1 Limitations to state explicitly

- Synthetic data cannot establish real-world effectiveness.
- Public datasets may differ from Indian UPI behaviour.
- The graph can only reconstruct relationships represented in supplied records.
- The system does not establish intent, guilt, legal liability, or guarantee recovery.
- The prototype does not execute real payments or access live UPI infrastructure.
- Advisories may be incomplete, stale, or affected by incorrect reports.
- Use of original records depends on authorisation and safeguards.

19. Expected Deliverables

- Working Trace.Pay application with payer and investigator interfaces.
- FastAPI backend with versioned REST routes and authenticated WebSocket updates.
- PostgreSQL schema, migrations, and seed scripts.
- Synthetic scenario generator with ground-truth labels.
- Validated ingestion pipeline and data-quality reports.
- Temporal graph service and multi-hop path reconstruction.
- Evidence provenance and gap-aware path summaries.
- Explainable rule engine and supporting transaction references.
- Sandbox QR flow and simulated payment lifecycle.
- RBAC, report review, correction/appeal workflow, and audit logging.
- Automated tests and reproducible evaluation harness.
- Data dictionary, dataset card, threat model, architecture and API documentation.

- Final report, presentation, demo script, and viva preparation.

20. Demonstration Script

- Open the sandbox and explain that the demo accounts are synthetic.
- Scan a sandbox recipient QR and request an advisory.
- Show an initial no-known-warning or insufficient-information state and its limitation.
- Start the synthetic event stream and show transfers appear in the graph and timeline.
- Open an alert and show its rule explanation and supporting transaction IDs.
- Open a path and inspect source references for observed edges.
- Switch to an incomplete-data scenario and show the path stops at the last supported transfer.
- Repeat the recipient check and show the updated advisory.
- Cancel or continue a simulated payment and show that no real transfer occurs.
- Finish with evaluation metrics, limitations, and future work.

21. Final Report Structure

Chapter	Contents
1. Introduction	Context, problem, motivation, objectives, scope
2. Literature review	Fraud detection, graph analytics, temporal tracing, evidence provenance
3. Problem formulation	Definitions, assumptions, threat model, research questions
4. System design	Architecture, data model, APIs, security, workflows
5. Methodology	Datasets, preprocessing, synthetic scenarios, algorithms, evidence model
6. Implementation	Stack, modules, interfaces, database, deployment
7. Evaluation	Setup, baselines, metrics, results, error analysis
8. Discussion	Interpretation, utility, limitations, generalisability
9. Conclusion and future work	Contributions, lessons, next steps
Appendices	API schemas, data dictionary, tests, setup guide, approved study materials

22. Literature Review and Research Gap

Build a literature matrix before asserting novelty. Record each source's citation, problem, data, method, output, evidence handling, evaluation, limitations, and relevance to Trace.Pay.

Field	Questions to answer
Citation	Authors, title, venue, year, DOI/stable URL
Data	Real or synthetic? Domain? Labels? Access conditions?
Method	Rules, graph analytics, GNN, tracing, provenance?
Output	Score, alert, path, report, payer warning?
Evidence handling	Are source records linked? Are gaps/uncertainty represented?
Evaluation	Metrics, baselines, protocol, limitations
Relevance	What is already solved and what remains to test?

Research-gap discipline Do not claim that no prior system supports a capability until the literature review tests that claim. A defensible contribution may be a careful integration, formalisation, or evaluation of related capabilities.

23. Candidate Research Contributions

Keep only contributions that are implemented and evaluated.

- A shared architecture linking payer-facing risk advisory to an investigator-facing temporal graph.
- Source provenance linking observed transaction edges to records and import metadata.
- A gap-aware path representation separating observed transfers from inferred relationships and unknown coverage.
- An explainable rule engine connecting alerts to supporting transactions and versioned rule definitions.
- A reproducible synthetic evaluation suite for known paths, incomplete records, duplicates, and benign lookalikes.
- An empirical comparison between ordinary graph tracing and tracing that explicitly exposes evidence gaps.

24. Future Work

- Evaluate authorised real-world records under a formal data-sharing arrangement.
- Conduct a larger usability study with an approved protocol.
- Explore probabilistic entity resolution with explicit confidence and human review.
- Compare rules with calibrated ML baselines if suitable labels and volume exist.
- Add graph storage or stream-processing infrastructure only if measured bottlenecks justify it.
- Study correction workflows and how advisory wording affects comprehension.
- Explore institutional integration only through formal partnerships and approved interfaces.

25. Glossary

Term	Meaning
Temporal graph	Graph where transaction timestamps and ordering matter
MultiDiGraph	Directed graph that allows multiple edges between a node pair
Fan-in	Multiple distinct senders transfer to one recipient in a defined period
Fan-out	One sender transfers to multiple recipients
Pass-through pattern	Incoming transfer followed by onward movement in a configured interval
Provenance	Information about the origin and handling of a record or result
Evidence gap	Part of the path cannot be checked using available records
Observed edge	Transaction represented by a validated source record
Inferred relationship	Analytical hypothesis not directly established by a source transaction
Idempotency	Repeated processing of the same event does not create duplicate effects
Risk advisory	Limited assessment to inform a decision, not establish guilt or guarantee safety
Ground truth	Known expected result in a controlled evaluation scenario
Pseudonymisation	Replacing direct identifiers with substitutes; data may still be linkable

26. Final Project Definition

Project in one paragraph Trace.Pay is a research prototype integrating explainable transaction-pattern detection, temporal graph-based multi-hop fund-flow reconstruction, source-record provenance, explicit evidence-gap reporting, and a sandbox pre-payment risk advisory. Its central experiment is whether evidence-aware path representation makes reconstruction more verifiable and transparent under controlled conditions. It uses documented synthetic scenarios and any suitable, permitted public or authorised datasets, reports results separately by data source, and avoids treating alerts or graph patterns as proof of fraud.

Keep this blueprint updated as implementation decisions are made. Mark each feature as Planned, In Progress, Implemented, Tested, or Deferred. Ensure the abstract, architecture, dataset card, and evaluation protocol describe the system you actually build.